

Eltropy Skill Builder - PRD

Mission Control for Agentic AI on the Front Office

Gaurav Mahto

1. Executive Summary

Eltropy has shipped the Main Orchestrator Agent and a first wave of Back Office sub-agents (Authentication, Account Services). The gap is the application layer that lets credit union AI Leads, Eltropy Forward Deployed PMs, and fintech partners author, scope, test, and publish new sub-agents and skills without engineering hand-offs.

The Skill Builder is that application - the authoring layer that turns the *App Store of banking* into reality. The unit of composition is the **Skill** (a Markdown SOP that calls **Tools**) bound to a **sub-agent** with closed, controlled agency. Skills are versioned, scoped, lint-able, replay-testable, and audit-logged before publish - because in a regulated space “AI can go nuts” is the failure mode the builder must prevent by design.

This PRD focuses on the Builder. The harness is out of scope for v1. **Grading metrics:** Guardrails. Usability. Affordance.

2. Mental Model

Sub-agent ::= role + intent-routing + skill bundle + guardrail policy
Skill ::= Markdown SOP + tool calls (curly-brace escape syntax)
Tool ::= named, schema-typed wrapper over one API

One Tool wraps one API. One Skill composes 1-N Tools in an MD SOP. One Sub-agent binds 1-N Skills within a scope (e.g. *loans-and-payments-and-services and nothing else*). The Main Orchestrator routes to Sub-agents the Builder publishes.

3. Personas

Persona	Tech fluency	What they do in the Builder
Eltropy Forward Deployed PM (the role I'd join)	Markdown + API + Postman + ships PRs	Authors Skills for a CU, ships to that tenant, feeds learning to platform team
CU AI Lead / Forward Deployed Engineer	API + Postman fluent; does not yet know what “Tool” means	Authors CU-specific Skills, tests, publishes to their own tenant. UI must teach Tools
Fintech Partner Skill Author	Engineer + product	Submits Skills to public Marketplace for review + distribution
CU Compliance Reviewer	Risk + audit	Read-only; reviews guardrail diffs, audit logs; approves publish

4. The Builder - five surfaces

4.1 Sub-agent Canvas. List + detail. Each sub-agent shows scope domain, bound skills, guardrail policy, version, last test pass. Scope-domain selection is a *bounded chip-picker* - the chip is the contract.

4.2 Skill Editor. Two-pane: Markdown left, live preview + tool-call validation right. Recognises `{{tool: ...}}` syntax inline:

1. Authenticate the member: `{{tool: authenticate_member(member_id=$member.id)}}`
2. If auth fails: `{{handback: orchestrator}}`
3. Otherwise: `{{tool: get_loan_status(loan_id=$loan.id)}}`
4. Summarise in 2 sentences. Stay within loan-servicing scope.

Inline autocomplete shows only Tools allowed in the bound sub-agent's scope - out-of-scope Tools cannot be summoned by typo. Live lint flags undefined Tool refs, missing params, scope violations, PII references. Tool calls render as **pills** in the preview so the SOP looks the way the runtime executes it.

4.3 Tool Registry. Each Tool: typed input/output schema, side-effect class (read / write / **financial**), required scopes, rate-limit tier, PII surface. Financial-side-effect Tools wear a persistent red badge. AI Leads do not author new Tools - they compose existing Tools into Skills.

4.4 Test Harness (replay sandbox). Paste a member transcript or pick from the last 50 production conversations (redacted). Runs in shadow mode with mocked Tool responses. Trace shows intent matched, sub-agent invoked, Skill called, Tool calls + arguments + returns, guardrails fired. **Affordance and Guardrails meet here** - the author sees the guardrail act before publish. Every Skill has a 5-50 case regression set; Skill cannot be published if any case breaks.

4.5 Publish & Versioning. Publish requires lints pass, regression set pass, and a version diff (including guardrail-policy diff) shown to the author. Compliance Reviewer approval on the tenant config. Published Skills are immutable. Sub-agents in production pin a Skill version.

5. Guardrails - first-class objects

Composed from named, reusable Guardrail Sets bound to a sub-agent's Guardrail Policy.

Family	Examples	Enforced at
Scope	Skill allowlist; sub-agent refuses + hands back on out-of-scope intent	Orchestrator + sub-agent self-check
Tool	Tool allowlist per Skill; financial Tools require step-up auth	Skill Editor lint + runtime
Data	PII redaction at log boundary; SSN never spoken on voice channel	Lint + runtime + audit
Behavioural	Refusal templates; escalation triggers; tone; never quote a balance pre-auth	Runtime
Audit	Every invocation logs Safe AI's four questions: what / why / what data / how	Runtime, append-only

Every published Skill carries a **Guardrail Policy Diff** in its version history. Compliance audits by sub-agent / Skill / Tool / member; exports logs in the format their NCUA examiner expects. This is the runtime expression of Safe AI's "AI never acts outside approved SOPs" tenet.

6. Usability & Affordance commitments

- **Time to first published Skill** (CU AI Lead, day 1): < 30 minutes, measured.
- **Bounded chip-pickers** instead of free-text fields wherever a contract exists.
- **Pill-rendered tool calls** so the SOP looks the way it executes.
- **Persistent red badge** on financial-side-effect Tools.
- **Drag-to-bind** for Skills onto sub-agents - visual matches the constraint.
- **"Why blocked?"** affordance on every lint error and blocked Tool call - the system explains itself.
- **One-click audit log** from any Skill or sub-agent.

7. Marketplace expansion - sequenced

Phase	Who publishes	Who installs	Why
0 (v1)	Elentropy	All tenants	Seed registry with Authentication + Account Services as reference
1	Elentropy + 3-5 design-partner CUs	Their tenant only	Validate authoring workflow on real CU workflows
2	All 750+ CUs	Their tenant only	GA. Skills stay tenant-scoped by default
3	Verified fintech partners (Constant AI / Skip-A-Pay style)	All tenants, opt-in	Open the App Store. Review pipeline: lint, guardrail audit, security review, certification badge
4	Open developer marketplace	All tenants, opt-in	Submission queue, dev sandbox tenant, public docs

Trust gradient maps to review-gate strictness. The App Store of banking earns trust the way Apple did - stricter, not faster, at the gate.

8. Out of scope for v1

Harness execution engine (Rahul's domain). Billing / revenue-share infrastructure (the *model* is \$9). Multi-tenant skill federation. Voice-channel authoring patterns (TTS, barge-in). A no-code "describe a Skill in English" generator.

9. The strategic ask - marketplace economics

§7 is the *what*. The economic model is *who profits, who pays, who therefore builds*. I'd want this aligned with you and the CPO inside my first two weeks - the publishing flow, certification tier, and partner contract template all branch on the answer.

My default POV (open to push-back):

Skill source	Pricing to CU	Elentropy take	Why
Elentropy baseline (~20-30 reference Skills)	Bundled in core subscription	n/a	Protects the <i>Financial Access For All</i> floor
CU-private (their AI Lead authors for their tenant)	Free	n/a	Their IP, their tenant
Verified fintech partner Skills (cross-tenant)	Metered per resolved conversation or flat add-on	20-25%	Salesforce AppExchange band - lower than Apple's 30%, higher than Stripe Connect's 3%; banking certification is heavyweight
Open developer marketplace	Author-set with a floor	15%	AWS Marketplace shape - volume over margin, lighter trust gate

Three sub-decisions to resolve before the prototype lands a Publish flow:

1. **Bundled vs metered for partner Skills.** Premium tier in the core subscription decides procurement once; metering stalls CFO forecasting. Lean: Premium tier + metered overage for high-volume Skills.
2. **Where Eltropy's 20-25% goes.** Lean: 70% to platform R&D, 30% into a **Skill Author Excellence Fund** that pays measured containment uplift back to top-performing CU AI Leads. Turns the AI Lead role into a career path inside CUs; flywheel like Stripe rewards top integrators.
3. **The free-floor commitment.** Authentication, Account Services, and any Skill that materially improves compliance posture (NCUA examiner readiness, fair-lending traceability) stay free across all CUs forever. The free floor is the moat against the regulatory backlash a paywalled-AI-in-banking story would invite.

The difference between a *certification badge* and a *certification gate* on a partner Skill is one toggle in the publishing flow - but the answer comes from you and the CPO, not from the prototype.

10. Success metrics

Dimension	Metric	v1 target
Adoption	CU AI Leads with at least one published Skill	20 in 90 days
Velocity	Median idea -> published Skill	< 60 minutes
Guardrails	Skills blocked pre-publish vs at runtime	> 90% caught pre-publish
Affordance	First-session task completion (publish first Skill) without help docs	> 70%
Trust	Compliance-reviewer approval cycle time	< 1 business day median

Appendix: Prototype plan (24h)

Built with Claude Code + Next.js + Tailwind. Hosted on Vercel. Shareable link.

Hour	Deliverable
0-2	Schema (Tool, Skill, Sub-agent, GuardrailSet, AuditEvent). SQLite seed with Authentication + Account Services as reference
2-6	Sub-agent Canvas + Skill Editor (Monaco + tool-call pill rendering)
6-10	Tool Registry + drag-to-bind + scope-aware autocomplete
10-14	Test Harness (mocked Tool responses; inline trace; guardrail outcomes)
14-18	Guardrail Policy editor + publish workflow + version diff
18-22	RBAC stub (role switcher); marketplace list view; Loom walkthrough
22-24	PRD review pass; submission

Bonus if time: wire one Sub-agent + Skill end-to-end against a stub harness.